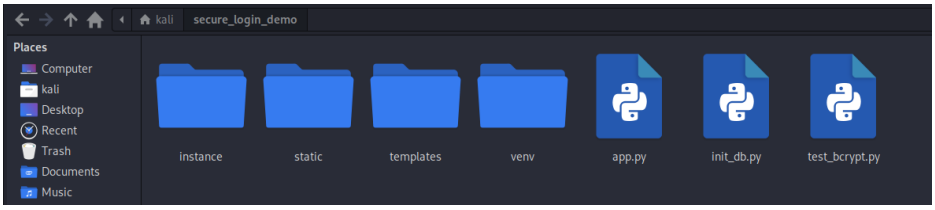



```
┌──(venv)─(kali㉿kali)-[~/secure_login_demo]
└─$ pip install flask bcrypt flask-sqlalchemy
Collecting flask
  Downloading flask-3.1.3-py3-none-any.whl.metadata (3.2 kB)
Collecting bcrypt
  Downloading bcrypt-5.0.0-cp39-abi3-manylinux_2_34_x86_64.whl.metadata (10 kB)
Collecting flask-sqlalchemy
  Downloading flask_sqlalchemy-3.1.1-py3-none-any.whl.metadata (3.4 kB)
Collecting blinker>=1.9.0 (from flask)
  Downloading blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
Collecting click>=8.1.3 (from flask)
  Using cached click-8.3.3-py3-none-any.whl.metadata (2.6 kB)
Collecting itsdangerous>=2.2.0 (from flask)
  Downloading itsdangerous-2.2.0-py3-none-any.whl.metadata (1.9 kB)
Collecting jinja2>=3.1.2 (from flask)
  Downloading jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
Collecting markupsafe>=2.1.1 (from flask)
  Using cached markupsafe-3.0.3-cp313-cp313-manylinux2014_x86_64.manylinux_2_17_x86_64.ma
l.metadata (2.7 kB)
Collecting werkzeug>=3.1.0 (from flask)
  Downloading werkzeug-3.1.8-py3-none-any.whl.metadata (4.0 kB)
Collecting sqlalchemy>=2.0.16 (from flask-sqlalchemy)
  Downloading sqlalchemy-2.0.49-cp313-cp313-manylinux2014_x86_64.manylinux_2_17_x86_64.ma
l.metadata (9.5 kB)
Collecting greenlet>=1 (from sqlalchemy>=2.0.16→flask-sqlalchemy)
  Downloading greenlet-3.4.0-cp313-cp313-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.
Collecting typing-extensions>=4.6.0 (from sqlalchemy>=2.0.16→flask-sqlalchemy)
```

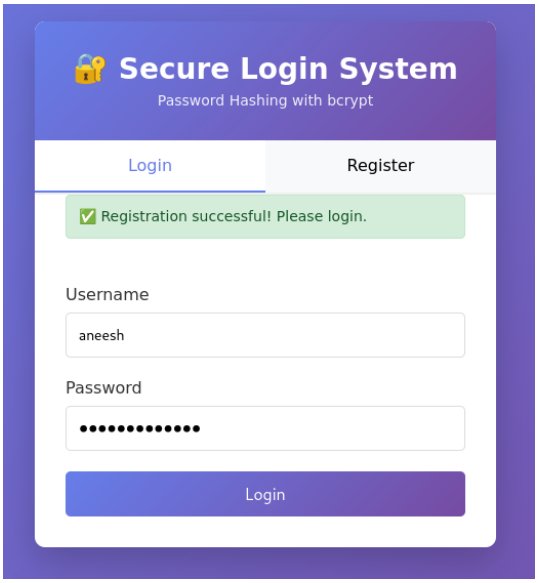
```
┌──(venv)─(root㉿kali)-[/home/kali/secure_login_demo]
└─# python3 test_bcrypt.py
═══════════════════════════════════════════════════════
BCRYPT PASSWORD HASHING DEMO
═══════════════════════════════════════════════════════

1. Original password: MySecretPassword123
2. Generated salt (random): b'$2b$12$CU6D7g9SAsS9J' ...
3. Final hash: $2b$12$CU6D7g9SAsS9JFBU5Dhmz.bDo2QEirlBnEiPoGq10.ZnpYZyAZoAy
   - Hash length: 60 bytes
   - Contains: algorithm, cost, salt, hash

4. Verification:
   Password 'MySecretPassword123' → ✓ Valid
   Password 'WrongPassword456' → ✗ Invalid

5. Key Security Features:
   ✓ Salt is random (rainbow table resistant)
   ✓ Adaptive algorithm (can increase cost factor)
   ✓ Slow by design (brute-force resistant)
   ✓ Hash includes salt + cost factor
```

```
┌──(venv)─(root㉿kali)-[/home/kali/secure_login_demo]
└─# python3 app.py
═══════════════════════════════════════════════════════
🔐 SECURE LOGIN SYSTEM STARTED
═══════════════════════════════════════════════════════
📍 Access at: http://127.0.0.1:5000
🔑 Test credentials: john_doe / SecurePass123
═══════════════════════════════════════════════════════
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it i
nstead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
═══════════════════════════════════════════════════════
🔐 SECURE LOGIN SYSTEM STARTED
═══════════════════════════════════════════════════════
📍 Access at: http://127.0.0.1:5000
🔑 Test credentials: john_doe / SecurePass123
═══════════════════════════════════════════════════════
 * Debugger is active!
 * Debugger PIN: 751-943-342
```



```
┌──(kali㉿kali)-[~/secure_login_demo]
└─$ sqlite3 instance/users.db
SQLite version 3.46.1 2024-08-13 09:16:08
Enter ".help" for usage hints.
sqlite> .tables
user
sqlite> SELECT username, password_hash FROM user;
john_doe|$2b$12$JC0..VJcW/2j2LBSnZywkOdj5WbaKU7bqqA6HYC7RfnWApkTcHWfy
aneesh|$2b$12$RjWsvogy9dlggoGzUG6a0OP3gPi9a.5e4gwXLQ8nBGE1LXNfSnexa
sqlite> .quit
```